API reference

The server exposes a JSON API under `/api`. The React app is its only intended client, but it is plain HTTP and can be used from scripts.

Conventions

- **Base URL:** `http://localhost:4000/api` in development.
- **Authentication:** every route except `GET /api/health` and `POST /api/auth/login` needs `Authorization: Bearer <token>`. Tokens come from login and last 8 hours. The user is looked up on every request, so a deactivated user is locked out at once.
- **Roles:** `admin` and `staff`. Routes marked *admin* return `403` to staff.
- **Money** is always **integer paise** (1 rupee = 100 paise), in fields ending `_paise`. Example: ₹2,500.50 is `250050`.
- **Dates** are `YYYY-MM-DD`. Timestamps are ISO 8601.
- **Lists** are paged: `?page=1&limit=20` (limit 1 to 100, default 20) and return `{ "items": [...], "total": 57, "page": 1, "limit": 20 }`.
- **Errors** are `{ "error": "A sentence a person can act on" }`. Status codes:

Status	Meaning
400	Validation failed (the message names the field and the rule)
401	Not logged in, token expired, or account deactivated
403	Admin access required
404	Not found
409	A unique value already exists (for example a username)
413	Upload too large (logo over 1 MB)
415	Wrong file type for an upload
429	Too many failed logins (10 per username and address per 15 minutes)
500	Unexpected server error (details are in the server log, never in the response)

Health

Method and path	Description
`GET /api/health`	`{ "ok": true }`. No login needed.

Authentication

Method and path	Body	Result
`POST /api/auth/login`	`{ "username", "password" }`	`{ "token", "user": { id, username, full_name, role } }`
`GET /api/auth/me`		`{ "user": { ... } }`

Users (admin)

Method and path	Body	Notes
`GET /api/users`		`{ "items": [users] }`
`POST /api/users`	`username`, `password`, `full_name`, `role` (default `staff`)	201. Username: 3-30 chars of `a-z 0-9 . _ -` (lower-cased). Password 8-100 chars.
`PATCH /api/users/:id`	any of `full_name`, `role`, `is_active`, `password`	You cannot deactivate or demote yourself.

Clients

Method and path	Description
`GET /api/clients?q=&include_deleted=&page=&limit=`	Search name, phone and referred-by. Deleted clients are hidden unless `include_deleted=true`.
`GET /api/clients/:id`	One client.
`POST /api/clients`	Create (201).
`PUT /api/clients/:id`	Replace the editable fields.
`DELETE /api/clients/:id`	**Soft** delete (204). Bills are kept.
`POST /api/clients/:id/restore`	Undo a delete.

Client fields and limits: `name` (required, 150), `phone` (20), `email` (150, must look like an address), `address` (300), `city` (100), `pan` (10, upper-cased), `gstin` (15, upper-cased), `referred_by` (150). Empty text is stored as `null`. Responses also include `folder_name` (see *client folders*), `is_deleted`, `created_at`, `updated_at`.

Services

Method and path	Description
`GET /api/services?include_inactive=`	Active services unless `include_inactive=true`.
`POST /api/services`	Create (201).

Method and path	Description
`PUT /api/services/:id`	Replace. Existing bills are not changed.

Fields: `name` (required, 150), `description` (500), `price_paise` (integer, 0 to 100,000,000,000), `is_active` (default `true`).

Bills

Method and path	Description
`GET /api/bills/financial-years`	`{ "items": [{ "fy_start": 2026, "label": "2026-27" }] }`: the years that have bills (newest first), always including the current one.
`GET /api/bills?q=&client_id=&fy=&page=&limit=`	Newest first. `q` matches the client name or bill number. `fy` is the **start year** of an Indian financial year (April to March): `fy=2025` means 1 Apr 2025 to 31 Mar 2026.
`GET /api/bills/:id`	The bill with `items`, client details, `fy_start`, and who created and last edited it.
`POST /api/bills`	Create (201).
`PUT /api/bills/:id`	Replace the client, date, notes and items.

Body for create and replace:

```
{
  "client_id": 12,
  "bill_date": "2026-11-05",
  "notes": "Optional, up to 1000 characters",
  "items": [
    {
      "service_id": 3,
      "description": "Statutory Audit",
      "details": "Optional text printed under the title, up to 500 characters",
      "quantity": 1,
      "unit_price_paise": 2500000
    }
  ]
}
```

- `bill_date` is optional (today when omitted). `service_id` and `details` are optional. `description` is 1 to 300 characters; `quantity` is a whole number 1 to 100,000; `unit_price_paise` is 0 to 100,000,000,000.
- 1 to 100 items. The items may add up to at most 1,000,000,000,000 paise (₹100 crore), otherwise `400`.
- The client must exist and not be deleted (an existing bill may keep a client deleted since, but cannot be moved to one).
- **GST is a snapshot.** On create the current setting is read once and stored as `gst_rate_percent` and `gst_paise` on the bill. On replace the bill's own stored rate is used, never the current setting.
- `bill_no` is assigned on create (`INV-000001`, ...) and never changes.
- Totals are returned as `subtotal_paise`, `gst_paise`, `total_paise`, and each line has `line_total_paise`.

Settings

Method and path	Who	Description
`GET /api/settings`	any user	Firm, payment and GST settings, plus `has_logo` and `logo_version`.
`PUT /api/settings`	admin	Replaces **all** of the fields below (send the whole form).
`GET /api/settings/logo`	any user	The logo image. `404` if none. Needs the token, so fetch it with the header rather than an `<img src>`.
`PUT /api/settings/logo`	admin	The image as the **raw request body** with `Content-Type: image/png`, `image/jpeg` or `image/webp`. Up to 1 MB.
`DELETE /api/settings/logo`	admin	Removes the logo.

Settings fields and limits: `org_name` (150), `org_address` (300), `org_gstin` (15, upper-cased), `bank_account_holder` (150), `bank_name` (150), `bank_account_number` (30), `bank_ifsc` (15, upper-cased), `bank_branch` (150), `upi_id` (100), `payment_instructions` (500), `gst_enabled` (boolean, required), `gst_rate_percent` (0 to 100, required).

The logo's real type is read from the file's first bytes, not the `Content-Type` header. SVG is refused because it can contain scripts.

Client folders (admin)

Method and path	Description
`GET /api/client-files/status`	`{ enabled, dir, lastError, lastSync }`
`POST /api/client-files/rebuild`	Rewrites every client's folder from the database. Returns the count and the status.

Example

```
# log in
TOKEN=$(curl -s -X POST http://localhost:4000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"YOUR-PASSWORD"}' | node -pe
'JSON.parse(require("fs").readFileSync(0)).token')

# bills for financial year 2025-26 matching "patel"
curl -s "http://localhost:4000/api/bills?q=patel&fy=2025" -H "Authorization: Bearer $TOKEN"
```